CSP-203: Software Tools & Technologies Lab

Chapter – 1

Basic Linux Commands (Lecture – 2)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Text Processing Commands

echo – display line of text

cat file.txt– Display file content

head -n 5 file.txt – Output the first parts of a file

tail -n 5 file.txt - Output the last parts of a file

cmp – Compare two files byte for byte

diff – Compare two text files line by line

more- allows you to scroll through the file

less – Lets you scroll up and down by pages or lines

Text Processing:

touch students.txt // create file student.txt

nano students.txt // open with nano

Type:

101 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C

Ctrl + o // To save

Ctrl + X // To exit from nano

Text Processing:

- **1.echo** – Print text to screen
  - echo "Welcome to Linux Text Processing"
- **2. cat** – Display file contents
  - cat students.txt
- **3. head / tail** – View top or bottom lines
  - head -n 2 students.txt # First 2 lines
  - tail -n 2 students.txt # Last 2 lines

```
anand@DESKTOP-OIRH1IL:~/STTL$ cat file1.txt
101 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C
anand@DESKTOP-OIRH1IL:~/STTL$ head -n 2 file1.txt
101 John A
102 Alice B
anand@DESKTOP-OIRH1IL:~/STTL$ tail -n 2 file1.txt
104 David B
105 Rohit C
```

Utilities and commands : head

head [OPTION] FILE_NAME

head syntax

show the specified number of lines

head –n 2 example.txt

```
anand@DESKTOP-OIRH1IL:~/STTL$ head -n 2 file1.txt
101 John A
102 Alice B
```

show the specified number of bytes

head –c 20 example.txt

```
anand@DESKTOP-OIRH1IL:~/STTL$ head -c 20 file1.txt
101 John A
102 Aliceanand@DESKTOP-OIRH1IL:~/STTL$
```

```
anand@DESKTOP-OIRH1IL:~/STTL$ head --help
Usage: head [OPTION]... [FILE]...
Print the first 10 lines of each FILE to standard output.
With more than one FILE, precede each with a header giving the file name.

With no FILE, or when FILE is -, read standard input.

Mandatory arguments to long options are mandatory for short options too.
  -c, --bytes=[-]NUM       print the first NUM bytes of each file;
                             with the leading '-', print all but the last
                             NUM bytes of each file
  -n, --lines=[-]NUM       print the first NUM lines instead of the first 10;
                             with the leading '-', print all but the last
                             NUM lines of each file
  -q, --quiet, --silent    never print headers giving file names
  -v, --verbose            always print headers giving file names
  -z, --zero-terminated    line delimiter is NUL, not newline
      --help     display this help and exit
      --version  output version information and exit

NUM may have a multiplier suffix:
b 512, kB 1000, K 1024, MB 1000*1000, M 1024*1024,
GB 1000*1000*1000, G 1024*1024*1024, and so on for T, P, E, Z, Y.
Binary prefixes can be used, too: KiB=K, MiB=M, and so on.

GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
Report any translation bugs to <https://translationproject.org/team/>
Full documentation <https://www.gnu.org/software/coreutils/head>
or available locally via: info '(coreutils) head invocation'
anand@DESKTOP-OIRH1IL:~/STTL$
```

```
anand@DESKTOP-OIRH1IL:~/STTL$ head file1.txt
101 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C
anand@DESKTOP-OIRH1IL:~/STTL$
```

Utilities and commands : tail

tail [OPTION] FILE_NAME

tail syntax

show the specified number of lines

tail –n 4 example.txt

show the specified number of bytes

tail –c 20 example.txt

```
anand@DESKTOP-OIRH1IL:~/STTL$ tail file1.txt
101 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C
anand@DESKTOP-OIRH1IL:~/STTL$ tail -n 2 file1.txt
104 David B
105 Rohit C
anand@DESKTOP-OIRH1IL:~/STTL$ tail -c 20 file1.txt
David B
105 Rohit C
anand@DESKTOP-OIRH1IL:~/STTL$
```

```
anand@DESKTOP-OIRH1IL:~/STTL$ tail --help
Usage: tail [OPTION]... [FILE]...
Print the last 10 lines of each FILE to standard output.
With more than one FILE, precede each with a header giving the file name.

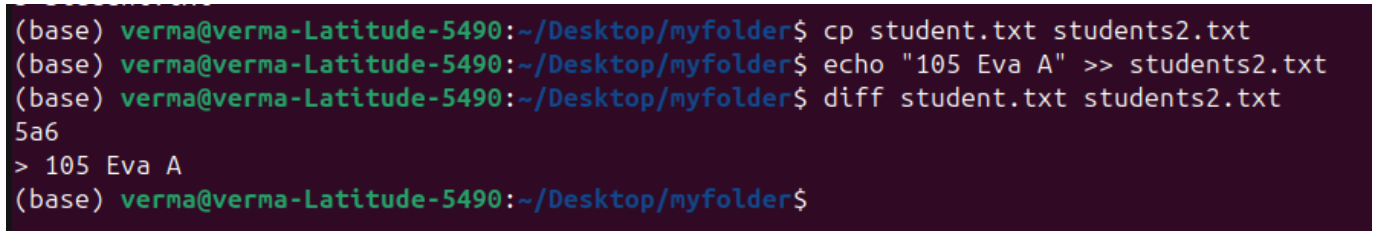
With no FILE, or when FILE is -, read standard input.

Mandatory arguments to long options are mandatory for short options too.
  -c, --bytes=[+]NUM       output the last NUM bytes; or use -c +NUM to
                             output starting with byte NUM of each file
  -f, --follow[={name|descriptor}]
                           output appended data as the file grows;
                             an absent option argument means 'descriptor'
  -F                       same as --follow=name --retry
  -n, --lines=[+]NUM       output the last NUM lines, instead of the last 10;
                             or use -n +NUM to output starting with line NUM
      --max-unchanged-stats=N
                           with --follow=name, reopen a FILE which has not
                             changed size after N (default 5) iterations
                             to see if it has been unlinked or renamed
                             (this is the usual case of rotated log files);
                             with inotify, this option is rarely useful
      --pid=PID            with -f, terminate after process ID, PID dies
  -q, --quiet, --silent    never output headers giving file names
      --retry              keep trying to open a file if it is inaccessible
  -s, --sleep-interval=N   with -f, sleep for approximately N seconds
                             (default 1.0) between iterations;
                             with inotify and --pid=P, check process P at
                             least once every N seconds
  -v, --verbose            always output headers giving file names
  -z, --zero-terminated    line delimiter is NUL, not newline
      --help     display this help and exit
      --version  output version information and exit
```

`diff` / `cmp` – Compare file

Create another file:

- cp students.txt students2.txt
- echo "105 Eva A" >> students2.txt
- diff students.txt students2.txt



Text Processing Commands

wc - Count lines, words, bytes in a file

sort - sort lines of text files

awk – A pattern scanning and processing language

grep – Print lines matching a pattern

grep <string> [file]

sort – Sort file content

sort students.txt

grep –

a. Search pattern

grep "B" students.txt

102 Alice B
104 David B

a. Case-insensitive search (-i)

grep -i "john" students.txt

101 John A

a. Display line numbers (-n)

grep -n "B" students.txt

2:102 Alice B
4:104 David B

a. Count matching lines (-c)

grep -c "B" students.txt op: 2

```
anand@DESKTOP-OIRH1IL:~/STTL$ cat file1.txt
106 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C
anand@DESKTOP-OIRH1IL:~/STTL$ sort file1.txt
102 Alice B
103 Bob C
104 David B
105 Rohit C
106 John A
anand@DESKTOP-OIRH1IL:~/STTL$ cat file1.txt
106 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C
```

```
anand@DESKTOP-OIRH1IL:~/STTL$ cat file1.txt
101 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C
anand@DESKTOP-OIRH1IL:~/STTL$ grep "b" file1.txt
103 Bob C
anand@DESKTOP-OIRH1IL:~/STTL$ grep "B" file1.txt
102 Alice B
103 Bob C
104 David B
anand@DESKTOP-OIRH1IL:~/STTL$
```

Utilities and commands : global regular expression print [grep]

grep [OPTION...] PATTERNS [FILE...] grep syntax

- grep searches for PATTERNS in each FILE

- Finds each line that matched the provided PATTERN

```
anand@DESKTOP-OIRH1IL:~/STTL$ grep --help
Usage: grep [OPTION]... PATTERNS [FILE]...
Search for PATTERNS in each FILE.
Example: grep -i 'hello world' menu.h main.c
PATTERNS can contain multiple patterns separated by newlines.

Pattern selection and interpretation:
  -E, --extended-regexp     PATTERNS are extended regular expressions
  -F, --fixed-strings       PATTERNS are strings
  -G, --basic-regexp        PATTERNS are basic regular expressions
  -P, --perl-regexp         PATTERNS are Perl regular expressions
  -e, --regexp=PATTERNS     use PATTERNS for matching
  -f, --file=FILE           take PATTERNS from FILE
  -i, --ignore-case         ignore case distinctions in patterns and data
      --no-ignore-case      do not ignore case distinctions (default)
  -w, --word-regexp         match only whole words
  -x, --line-regexp         match only whole lines
  -z, --null-data           a data line ends in 0 byte, not newline

Miscellaneous:
  -s, --no-messages         suppress error messages
  -v, --invert-match        select non-matching lines
  -V, --version             display version information and exit
      --help                display this help text and exit

Output control:
  -m, --max-count=NUM       stop after NUM selected lines
  -b, --byte-offset         print the byte offset with output lines
  -n, --line-number         print line number with output lines
      --line-buffered       flush output on every line
  -H, --with-filename       print file name with output lines
  -h, --no-filename         suppress the file name prefix on output
      --label=LABEL         use LABEL as the standard input file name prefix
  -o, --only-matching       show only nonempty parts of lines that match
  -q, --quiet, --silent     suppress all normal output
      --binary-files=TYPE   assume that binary files are TYPE;
                            TYPE is 'binary', 'text', or 'without-match'
  -a, --text                equivalent to --binary-files=text
```

Utilities and commands : grep (1)

```
grep [OPTION...] PATTERNS [FILE...]
```

grep syntax

Find a Matching String with grep

```
grep “unix” csp203.txt
```

How to ignore case distinctions

```
grep -i “UNIX” cs203.txt
```

csp203.txt

In this course, we will begin with an introduction to the Unix operating system. We will be using Linux and we will see how we can use the power of unix to manipulate the File System (FS) and use unix tools and shell scripting to accomplish interesting tasks. Our focus would be on the unix features that are more directly related to writing, debugging and maintaining C programs. We will also focus on unix shell scripting, so we can develop powerful scripts for managing tasks such as unix system calls, file manipulation etc. To find out which version of the operating system you are running type

```
anand@DESKTOP-OIRH1IL:~/STTL$ grep "unix" csp203.txt
In this course, we will begin with an introduction to the Unix operating system. We will be using Linux and we will see how we can use the power of u
nix to manipulate the File System (FS) and use unix tools and shell scripting to accomplish interesting tasks. Our focus would be on the unix feature
s that are more directly related to writing, debugging and maintaining C programs. We will also focus on unix shell scripting, so we can develop powe
rful scripts for managing tasks such as unix system calls, file manipulation etc. To find out which version of the operating system you are running t
ype
anand@DESKTOP-OIRH1IL:~/STTL$ grep -i "unix" csp203.txt
In this course, we will begin with an introduction to the Unix operating system. We will be using Linux and we will see how we can use the power of u
nix to manipulate the File System (FS) and use unix tools and shell scripting to accomplish interesting tasks. Our focus would be on the unix feature
s that are more directly related to writing, debugging and maintaining C programs. We will also focus on unix shell scripting, so we can develop powe
rful scripts for managing tasks such as unix system calls, file manipulation etc. To find out which version of the operating system you are running t
ype
```

e. Use regular expressions (. ^ $ etc.)

grep "^103" students.txt # Lines starting with 103

grep "C$" students.txt # Lines ending with C

f. Search recursively in files (-r)

grep -r "main" /home/user/code/

Searches the pattern "main" inside all files under the given directory.

wc – Word/line/char count

wc students.txt op: 4 12 48 students.txt

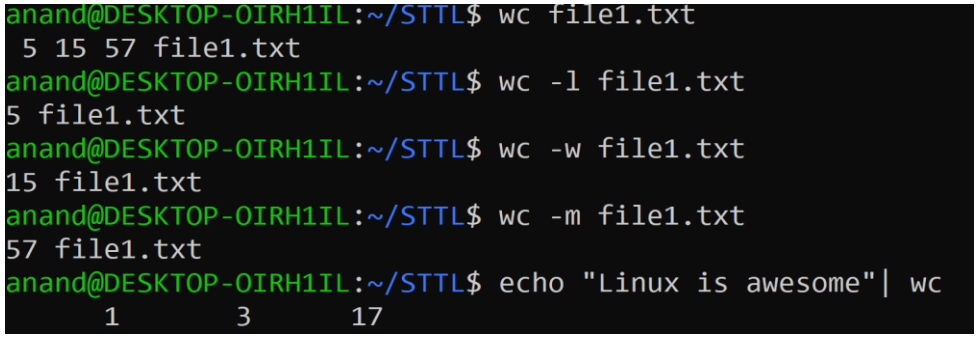
wc -l students.txt op: 4 students.txt //Count only lines

wc -w students.txt op: 12 students.txt //Count only words

wc -m students.txt op: 48 students.txt // Count only characters

Count from standard input:

echo "Linux is awesome" | wc op: 1 3 17



The echo command by default adds a newline character at the end of the string. This newline character is also counted by wc as a character.

awk – Advanced field processing

awk is a **field-based pattern scanning and processing language** used to process and analyze text files — especially those structured into columns (like CSV, tabular logs, etc.).

Basic Syntax: awk 'pattern { action }' filename

pattern: what to match (optional)

action: what to do when matched

Eg. awk '{print $2}' students.txt //Print the second column (Name)

awk '{print $1, $3}' students.txt //Print Roll Number and Grade

awk '{print $2 " got grade " $3}' students.txt // Add a custom message

awk '$3=="B" {print $2}' students.txt // Print only if grade is B (with if)

awk '{print NR, $0}' students.txt //Print line numbers

```
anand@DESKTOP-OIRH1IL:~/STTL$ cat file1.txt
101 John A
102 Alice B
103 Bob C
104 David B
105 Rohit C
```

```
anand@DESKTOP-OIRH1IL:~/STTL$ awk '{print $2}' file1.txt
John
Alice
Bob
David
Rohit
anand@DESKTOP-OIRH1IL:~/STTL$ awk '{print $1, $2}' file1.txt
101 John
102 Alice
103 Bob
104 David
105 Rohit
anand@DESKTOP-OIRH1IL:~/STTL$
```

Bash Pipe (“|”)

- Used to combine two or more commands
- The output of one command acts as input to another command
- It can also be visualized as a temporary connection between two or more commands/ programs/ processes.

Syntax: cmd1 | cmd2 | ..... | cmdN

Command	***Description***
cmd1 | cmd2	Stdout of cmd1 to cmd2

*cmd - command

Piping(|)

Bash scripting is all about combining simple commands together to do more powerful things. This is accomplished using the ”pipe” character

- <command1> | <command2>
  - ❑ Passes the output from command1 to input of command2

Eg.

- cat file.txt | wc -l # Count lines in file
- history | head -20
- history | head -20 | tail -10

Piping

What is a pipe?

A pipe | **takes the output of one command** and **uses it as input for another command**.

Basic Syntax: command1 | command2

1. **View top 5 lines of a long file**

```
cat file.txt | head -n 5
```

2. Count lines that contain a keyword

```
grep "text" file.txt | wc -l
```

3. **Find top 5 commands in history**

```
history | head -n 5
```

Processes and tasks management

ps – View current running process

top – Live, real-time process monitoring

pidof process_name – Find the PID of a running process

kill PID – Terminate a process by its PID

killall - Terminate all processes (in GNU/Linux, it's kill by name)

wait – Wait for Background Processes

Process Management Commands

1. **`ps` – Show current processes**

   ps // Shows processes in the current shell session.

   ps -aux // Lists all running processes with detailed info.

1. **`top` – Real-time system monitor**

   top // Shows dynamic list of running processes, CPU/mem usage.

   🔄 Press q to quit.

1. **`kill` – Terminate a process by PID**

   kill 12345

1. **`killall` – Kill by process name**

   killall firefox

1. **`pidof` – Get PID of a program**

   Pidof chrome

Networking Command:

ping – Check if a host is reachable and measure latency

ifconfig – View or configure network interfaces (deprecated, use ip)

ip a – Show IP addresses of network interfaces

nslookup – Query DNS to get IP of a domain

traceroute – Show the route packets take to a host

ssh – Secure shell remote login to another machine

wget – Download files from the internet

N/w Commands:

`ping` – Check network connectivity

ping google.com

`ifconfig` – Show or configure network interfaces

Ifconfig // Shows IP address, MAC address, interface status.

`ip a` – Modern IP address configuration tool

ip a // Lists all IP addresses and network interfaces.

`netstat` – Show network connections and routing

netstat -tuln // Shows active TCP/UDP listening ports.

`traceroute` – Trace route to a remote host

traceroute google.com // Displays the path taken by packets to reach destination.

`ssh` – Secure remote login

ssh user@hostname // Connects to a remote machine over SSH.

wget – Download files from the web

wget https://example.com/file.txt // Downloads file from a URL.

Text Editors in Linux

What is a Text Editor?

A software tool used to create and edit plain text files

Used heavily for:

- Writing code
- Editing config files
- Creating shell scripts

Types of Text Editors:

CLI-based - nano, vi / vim, emacs

GUI-based - gedit, Kate, Leafpad, VS Code, Sublime

Nano – Beginner Friendly Editor

Simple and user-friendly

Command to open: nano filename.txt

Common Shortcuts:

- Ctrl + O: Save
- Ctrl + X: Exit
- Ctrl + K: Cut line
- Ctrl + U: Paste line
- Ctrl + G: Get Help

Text Editors

nano – Simple & Beginner-Friendly Editor

Demo Steps:

1. Create/open a file: `nano notes.txt    // If the file doesn't exist, it is created.`
2. Type: `Hello, this is a demo.`
3. Save: `Ctrl+O`, then press `Enter`
4. Exit: `Ctrl+X`

Common Commands inside `nano`:

Ctrl+o Write/save file

Ctrl+X Exit

Ctrl+K Cut current line

Ctrl+U Paste

Ctrl+W Search

Ctrl+G Help

Vim – Powerful CLI Editor

What is vim?

- vim = Vi IMproved — a powerful and popular command-line text editor.
- Comes with advanced features like syntax highlighting, multiple modes, and scripting support.
- Preferred by system admins, developers, and power users.

How to Open a File in vim

Command: vim filename.txt

Three Main Modes in Vim

1. **Editing (normal) mode:**

➢ Launching pad to issue commands or go into other modes

➢ Allows you to view the text but not edit it

➢ Vim starts in normal mode

➢ You can jump to normal mode by pressing the Escape (Esc) key on your keyboard.

VIM Modes

2. Insert mode:

- ➢ Used to type text into the buffer (file)
- ➢ This probably what you're used to from your text editor
- ➢ You get to the insert mode by pressing the i key on your keyboard

3. Visual mode:

- ➢ Used to highlight text and perform operations on selected text
- ➢ You get to visual mode from normal mode by pressing the v key on your keyboard

VIM Commands (Basic)

Entering normal mode

Press <ESC>

Entering insert mode

<i>

Entering visual mode

<v>

VIM Commands (Basic)

Save file

:w

Exit

:q

Quit without saving

:q!

Vim Text Editor:

`vim` – Advanced Modal Editor (for skilled users)

Demo Steps:

1. Open: `vim notes.txt`
2. Press `i` → write: `Vim demo text`
3. Press `Esc`
4. Save and quit: `:wq`

Modes in Vim:

- **Normal mode**: Default (navigation, delete, etc.)
- **Insert mode**: For typing text (`i`, `a`, `o`)
- **Command mode**: Start with `:` (e.g. `:w`, `:q`)

Vim Text Editor:

➤ Common Commands:

Key	Action
i	Insert mode (start editing)
Esc	Back to normal mode
:w	Save
:q	Quit
:wq	Save and quit
dd	Delete a line
u	Undo
/text	Search text

Thank you!

Email: anand@iitbhilai.ac.in